DOABA INSTITUTE OF ENGINEERING AND TECHNOLOGY

Kharar, Mohali

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

A PROJECT REPORT ON

HOUSE PRICE PREDICTION SYSTEM USING MACHINE LEARNING AND FLASK

Submitted in partial fulfillment of the requirements for

45 DAYS INDUSTRIAL / SUMMER TRAINING

SUBMITTED TO:

Department of CSE
DIET, Kharar, Mohali

SUBMITTED BY:

Name: Amarjit Singh
Roll No: 2428578
Branch: CSE

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to **Doaba Institute of Engineering and Technology (DIET), Kharar, Mohali**, and the **Department of Computer Science and Engineering** for providing me with the opportunity and state-of-the-art facilities to undertake and complete this 45-day industrial training project.

I am deeply thankful to my project guide and all faculty members of the CSE Department whose invaluable guidance, continuous support, and constructive feedback made the successful completion of this project possible.

Finally, I would like to extend my heartfelt thanks to my peers and family for their unwavering support and encouragement throughout the research and development phases of this software application.

(Amarjit Singh)
Roll No: 2428578
Department of Computer Science & Engineering
DIET, Kharar, Mohali

ABSTRACT

Accurate real estate valuation is crucial for home buyers, property sellers, and financial institutions. Traditional property valuation techniques often rely on manual appraisals, which can be subjective, time-consuming, and prone to human bias or error. This project presents an automated, web-based **House Price Prediction System** that utilizes Machine Learning algorithms integrated with a lightweight Flask web framework to estimate residential property prices dynamically based on structural and architectural parameters.

The core machine learning pipeline processes raw dataset features—including lot dimensions, basement square footage, building category, and construction year—using `pandas` and `scikit-learn`. Categorical variables are transformed via One-Hot Encoding, and numeric features are normalized using `StandardScaler`. A supervised `RandomForestRegressor` algorithm was trained on the preprocessed dataset to capture non-linear feature dependencies effectively.

To deliver an accessible solution for non-technical end-users, an intuitive full-stack web interface was developed using HTML5, CSS3, and Python Flask. The backend exports model artifacts (`model.pkl`, `scaler.pkl`, `model_columns.pkl`) to execute real-time inference on feature inputs submitted through the interactive client UI.

TABLE OF CONTENTS

1. Introduction

1.1 Project Overview | 1.2 Problem Statement | 1.3 Objectives

2. System Requirements

2.1 Hardware Requirements | 2.2 Software Requirements

3. System Architecture & Methodology

3.1 Data Preprocessing Pipeline | 3.2 Machine Learning Model | 3.3 Flask Workflow

4. Implementation Details

4.1 Directory Structure | 4.2 Model Artifacts Description

5. Conclusion & Future Scope

5.1 Conclusion | 5.2 Future Enhancements

6. References

1. INTRODUCTION

1.1 Project Overview

The real estate market plays a vital role in the global economy. Valuing residential real estate accurately is essential for financial institutions, realtors, investors, and prospective buyers. Machine Learning (ML) techniques provide powerful frameworks capable of processing complex datasets, identifying hidden trends, and delivering objective property price forecasts.

1.2 Problem Statement

Determining a fair market price for residential properties requires evaluating multiple interdependent parameters (e.g., lot square footage, basement area, overall quality ratings, year of construction, zoning codes). Traditional appraisal processes are slow, manual, and inconsistent. There is a pressing need for an automated, data-driven software platform that processes user inputs via a web form and immediately returns a precise monetary evaluation.

1.3 Objectives

- To design and implement a machine learning pipeline capable of preprocessing, scaling, and training on real estate features.
- To train a high-performance `RandomForestRegressor` model for precise numerical target estimation.
- To construct an interactive web user interface using HTML5 and CSS3 for user data entry.
- To integrate the trained model behind a robust Flask backend server to deliver instant predictions.

2. SYSTEM REQUIREMENTS

2.1 Hardware Requirements

Component	Minimum Specification	Recommended Specification
Processor	Dual-core 2.0 GHz (Intel i3 / AMD Ryzen 3)	Quad-core 2.8 GHz or higher (Intel i5 / Ryzen 5)
System Memory (RAM)	4 GB	8 GB or higher
Disk Storage	500 MB free space	2 GB free space
Display Resolution	1024 x 768	1920 x 1080 (Full HD)

2.2 Software Requirements

Category	Tool / Specification
Operating System	Windows 10 / 11, macOS, or Ubuntu Linux
Programming Language	Python 3.10+
Development Environment	Visual Studio Code (VS Code)
Core Python Libraries	`flask`, `scikit-learn`, `pandas`, `numpy`, `joblib`
Web Browser	Google Chrome, Mozilla Firefox, or Microsoft Edge

3. SYSTEM ARCHITECTURE & METHODOLOGY

The system is split into two main functional modules: the **Offline Training Pipeline** and the **Online Prediction Server**.

3.1 Data Preprocessing Pipeline

Raw input data contains missing values, continuous variables, and categorical fields that cannot be directly digested by mathematical regression models:

- **Imputation:** Numerical features missing values are replaced with column median values; missing categorical fields are labeled as `'Missing'`.
- **One-Hot Encoding:** Discrete fields (such as `MSZoning`, `BldgType`) are split into binary columns using `pandas.get_dummies()`.
- **Standard Scaling:** Continuous features are transformed using scikit-learn's `StandardScaler`:

$$z = (x - \mu) / \sigma$$

Where μ represents feature mean and σ represents standard deviation.

3.2 Machine Learning Model

A `RandomForestRegressor` was chosen due to its resilience against overfitting and ability to handle non-linear relationships. The ensemble model combines multiple decision trees trained on random bootstrap sub-samples of the dataset.

3.3 Flask Integration Workflow

```
[ Client Browser ] ———( Submits Web Form )———▶ [ Flask app.py Server ]
                                                          |
                                                Loads model.pkl & scaler.pkl
                                                          |
[ Displays Result ] ◀——( Renders index.html )——— Calculates Prediction
```

4. IMPLEMENTATION DETAILS

4.1 Project Directory Structure

```
house_price_app/
│
├── train.py                # Model training and artifact serialization script
├── app.py                  # Main Flask application backend server
├── model.pkl               # Serialized Random Forest model
├── scaler.pkl              # Serialized StandardScaler object
├── model_columns.pkl       # Serialized list of feature columns
│
├── static/
│   └── style.css           # Modern layout stylesheet
│
└── templates/
    └── index.html          # User interactive input form template
```

4.2 Saved Artifact Definitions

- `model.pkl`: Stores the trained Random Forest decision trees.
- `scaler.pkl`: Preserves the exact mean and variance values computed during training to scale new user inputs correctly.
- `model_columns.pkl`: Holds the exact list and order of one-hot encoded columns to align user inputs before running model inference.

5. CONCLUSION & FUTURE SCOPE

5.1 Conclusion

The **House Price Prediction System** effectively connects advanced Machine Learning algorithms with an accessible Web UI using Python and Flask. By encapsulating automated preprocessing, standard scaling, and One-Hot alignment into saved model artifacts, the application delivers rapid, consistent, and accurate property valuations to end users.

5.2 Future Enhancements

- **Geographic & Map Integration:** Incorporate live Google Maps API data to automatically evaluate neighborhood ratings and local amenities.
- **Deep Learning Extensions:** Experiment with Multi-Layer Perceptrons (MLP) and Gradient Boosting frameworks (XGBoost/LightGBM) for improved model performance.
- **Cloud Deployment:** Deploy the web application on cloud infrastructure like AWS, Azure, or Render for public deployment.

6. REFERENCES

1. Scikit-learn: *Machine Learning in Python*, Pedregosa et al., JMLR 12, pp. 2825-2830, 2011.
2. Flask Documentation: *Pallets Web Development Ecosystem* (https://flask.palletsprojects.com/).
3. McKinney, W. (2010). *Data Structures for Statistical Computing in Python*. Proceedings of the 9th Python in Science Conference.
4. Breiman, L. (2001). *Random Forests*. Machine Learning, 45(1), 5-32.